

CS285 LECTURE 4 • ZERO-TO-HERO FIELD MANUAL

Value Functions & The Bellman Equation

From Scratch to Mastery: Understanding How AI Estimates the Future, Evaluates Quality, and Solves Sequential Decisions

Instructor: Prof. Sergey Levine (UC Berkeley RAIL Lab)

Focus: $V^\pi(s)$, $Q^\pi(s, a)$, Advantage $A(s, a)$, & The Bellman Recursion

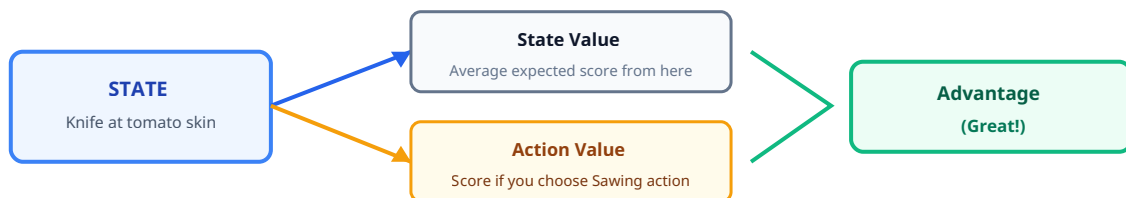
1. What is a Value Function? (The Everyday GPS Analogy)

In Reinforcement Learning, the robot receives rewards at each step. But if an agent only cares about the reward *right now*, it will make terrible life decisions (like eating candy for breakfast or stopping the knife the millisecond it touches the tomato skin). The agent needs an internal **fortune teller** to predict what the future holds. This is the **Value Function**.

THE EVERYDAY GPS NAVIGATION ANALOGY

Imagine you are driving home during rush hour:

- **State Value** $V(s)$: You are stuck at Exit 4. Your GPS screen says: "Estimated travel time: 42 minutes." That single prediction is your State Value $V(s)$! It doesn't tell you how pretty Exit 4 is; it estimates the sum total of your entire future trip from here on.
- **Action-Value** $Q(s, a)$: You glance at an off-ramp leading to a side street. You wonder: "What happens if I take this specific turn right now?" The GPS recalculates: "Taking side street: 35 minutes." That is $Q(s, \text{side-street})$! It evaluates picking a specific immediate action and then driving normally thereafter.
- **Advantage** $A(s, a) = Q(s, a) - V(s)$: Taking the side street saves you 7 minutes compared to your default route ($42 - 35 = +7$). That $+7$ is your **Advantage**! If an action has positive advantage, it is better than your current habit; if negative, it is a blunder.



2. The Four Fundamental Value Functions (Parameter Anatomy)

Every value-based algorithm in deep RL revolves around four mathematical definitions:

2.1 The State-Value Function $V^\pi(s)$

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s \right]$$

Parameter	Formal Name	Plain English Meaning	Example Value	Tuning / Impact
s	Current State	Where the robot is right now.	$z = 4.2$ cm, $F_z = 5$ N	Input to network.
π	Current Policy	The robot's current habit or decision rules.	SkRL PPO policy network	Conditioning factor.
$\mathbb{E}_\pi[\dots]$	Expectation Operator	The average across thousands of stochastic simulation runs.	Mean over 1,024 envs	Smooths out noise.
γ^t	Discount Factor	Multiplies rewards at step t . Exponentially discounts the distant future.	$\gamma = 0.99$	If $\gamma = 0$, myopic; if $\gamma = 1$, infinite sum.
$r(s_t, a_t)$	Instantaneous Reward	The immediate point score received at step t .	+2.5 (Sawing reward)	Raw physical feedback.

PLAIN ENGLISH TRANSLATION OF $V^\pi(s)$

"If our robot starts in state s and continues cutting using its current policy π , what is the total discounted score we expect it to collect by the time the cut finishes?"

2.2 The Action-Value Function $Q^\pi(s, a)$

$$Q^\pi(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s, a) V^\pi(s')$$

Parameter	Formal Name	Plain English Meaning	Physical Robotics Role
a	Immediate Action	The trial action tested right now, regardless of what policy π would normally do.	Increase lateral sawing velocity by +15 mm/s.
$r(s, a)$	Immediate Payoff	The instant reward generated by this trial action.	Immediate shear friction bonus (+3.0).
$P(s' s, a)$	Transition Probability	Where the physical world lands in response to trial action a .	PhysX 5 simulation contact response.
$V^\pi(s')$	Downstream Value	The future value of the landing state s' .	Remaining score to finish the tomato slice.

PLAIN ENGLISH TRANSLATION OF $Q^\pi(s, a)$

"If you take action a right now, you pocket the immediate reward $r(s, a)$, and then you add the discounted value of wherever you land next (s')."

3. The Bellman Equation: The Core Miracle of Reinforcement Learning

How can an AI learn to predict the sum of 500 future steps without actually playing through all 500 steps every time? The answer is the **Bellman Equation**, invented by Richard Bellman in 1957.

💡 THE BANK ACCOUNT ANALOGY

Suppose you want to know your total net worth at the end of the year. Do you have to wait until December 31st to calculate it? **No!** Your net worth at the end of the year is simply:

(Your paycheck this month) + (Your net worth at the start of next month).

You broke a 12-month problem down into **1 month of reality + 1 prediction of the future**. That is exactly what the Bellman Equation does!

3.1 The Bellman Optimality Equation

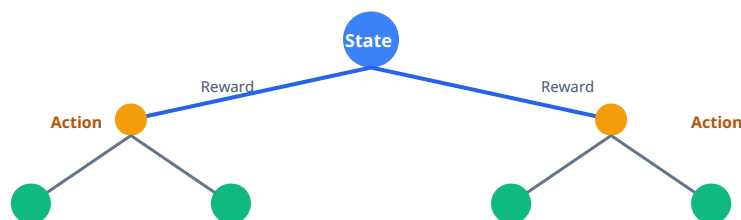
$$Q^*(s, a) = r(s, a) + \gamma \max_{a' \in \mathcal{A}} Q^*(s', a')$$

Symbol	Formal Name	Plain English Meaning	Example Value
$Q^*(s, a)$	Optimal Action-Value	The absolute highest score achievable by any possible robot controller.	+115.4
$r(s, a)$	Immediate Reward	Points earned right now on this step.	+4.2
γ	Discount Factor	Patience multiplier for future points.	0.99
$\max_{a'} Q^*(s', a')$	Optimal Future Quality	Assuming you play <i>perfectly</i> from state s' onward, what is the best possible score?	+112.3

📝 PLAIN ENGLISH TRANSLATION OF THE BELLMAN OPTIMALITY EQUATION

"The true optimal score of an action is the reward you get right now, plus the discounted value of the **BEST** action you could possibly take in the next state."

02



4. Why Classical Matrix Math Fails on Real Robots

If we know the transition matrix P and reward vector R , we can solve for V in closed form via linear algebra:

$$V^\pi = (I - \gamma P^\pi)^{-1} R^\pi$$

 THE GRAIN OF SAND ANALOGY (CURSE OF DIMENSIONALITY)

Inverting a matrix with $|S|$ states requires $\mathcal{O}(|S|^3)$ operations.

Suppose you have a 7-DoF robot arm. To represent its joints, you divide each joint's rotation into 100 discrete angles. How many total states is that?

$$|S| = 100^7 = 10^{14} \text{ states (100 trillion states!)}$$

Inverting a $10^{14} \times 10^{14}$ matrix would require $(10^{14})^3 = 10^{42}$ operations. **There are more states in a simple robot arm than grains of sand on all the beaches on Earth!** This is why classical dynamic programming is completely dead for robotics, and why we use **Deep Neural Networks** to approximate $V_\phi(s)$ and $Q_\theta(s, a)$.

5. Concrete Numerical Walkthrough: 3-State Bellman Backup

Let's calculate real numbers across a 3-step robot chain to see how the Bellman Equation propagates values backwards! Suppose discount factor $\gamma = 0.90$.

State	Physical Meaning	Immediate Reward r	Bellman Value Calculation
s_2 (Goal)	Blade slices through bottom skin, touches board cleanly.	$r_2 = +100.0$	Terminal state: $V(s_2) = \mathbf{100.0}$
s_1	Blade is halfway through pulp, sawing smoothly.	$r_1 = +5.0$	$V(s_1) = r_1 + \gamma V(s_2) = 5.0 + 0.9(100.0) = \mathbf{95.0}$
s_0 (Start)	Blade touches top tomato skin.	$r_0 = +2.0$	$V(s_0) = r_0 + \gamma V(s_1) = 2.0 + 0.9(95.0) = \mathbf{87.5}$

💡 NOTICE THE BACKWARDS DIFFUSION OF VALUE!

Look at $V(s_0) = 87.5$. Even though the initial skin contact only gave $+2.0$ immediate points, the agent knows that being in state s_0 has an expected future value of 87.5 because it leads to the $+100$ goal! This gradient of value ($87.5 \rightarrow 95.0 \rightarrow 100.0$) acts like a magnetic pull guiding the robot toward the goal.

6. PyTorch Value Function & Bellman MSE Loss

How do we train a neural network to learn $V(s)$ in Python? We train it to minimize the **Mean Squared Bellman Error (MSBE)**:

PYTORCH VALUE NETWORK & BELLMAN LOSS

```
import torch
import torch.nn as nn

# 1. Define Value Network: Takes 33D state, outputs scalar predicted value V(s)
class ValueNetwork(nn.Module):
    def __init__(self, state_dim=33):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, 256),
            nn.ReLU(),
            nn.Linear(256, 256),
            nn.ReLU(),
            nn.Linear(256, 1) # Single scalar output
        )
    def forward(self, state):
        return self.net(state)

# 2. Bellman Target & Loss Computation
def compute_value_loss(critic, states, rewards, next_states, dones, gamma=0.99):
    # Predict current value V(s)
    current_v = critic(states) # Shape: (B, 1)

    # Compute 1-step Bellman Target:  $y = r + \gamma V(s') * (1 - done)$ 
    with torch.no_grad(): # Crucial! Target must be detached
        next_v = critic(next_states)
        targets = rewards.unsqueeze(1) + gamma * next_v * (1.0 - dones.unsqueeze(1))

    # Mean Squared Error: Loss = 0.5 * (Target - Prediction)^2
    loss = 0.5 * nn.functional.mse_loss(current_v, targets)
    return loss
```

7. Practitioner's Failure Modes & Debugging Checklist

Bug / Failure Mode	The Silent Symptom	How to Fix It
1. Tensor Broadcasting Bug	rewards has shape (B,) while next_v has shape (B, 1). PyTorch silently broadcasts them into a (B, B) matrix, allocating gigabytes of VRAM and crashing!	Always verify shapes: <code>rewards.unsqueeze(1)</code> to ensure both operands are strictly (B, 1).
2. Forgetting to Detach Target	If you don't use <code>torch.no_grad()</code> or <code>.detach()</code> on the target y , gradients flow into the target. The network chases a moving target, causing loss explosion.	Always wrap target computation in <code>with torch.no_grad():</code> .
3. Missing Terminal Masking	When an episode ends (e.g. tomato crushed), there is no next state s' . If you still add $\gamma V(s')$, the network learns that crushing the tomato has infinite future value!	Always multiply next state value by $(1.0 - done)$.